

National University of Computer and Emerging Sciences

FAST-NU, Islamabad Campus

Data Warehousing & Business Intelligence

Semester Project Report



Submitted By

Name: Eeshaal Adeel

NU ID: 23I-2500 / BDS-5C

Submitted To

Ma'am Azka Atiq

Department of Data Science

Dated: November 17th, 2025

1. Project Overview

This project focuses on designing, implementing, and analyzing a near real-time Data Warehouse (DW) prototype for Walmart, one of the world's largest retail chains. The primary objective is to create a system capable of efficiently integrating and processing transactional data to enable timely business insights and decision-making.

1.1 Business Context

Walmart serves millions of customers daily across its stores and online platforms. To optimize sales and enhance customer satisfaction, the company needs to analyze shopping behavior in near real-time, enabling dynamic promotions and personalized offers. This requires a robust data warehousing solution that can handle high-volume transactional data streams while maintaining data quality and enabling complex analytical queries.

1.2 Project Objectives

The main objectives of this project include:

1. **Design a Star Schema:** Create an appropriate dimensional model for Walmart's sales data that supports efficient querying and analysis.
2. **Implement HYBRIDJOIN Algorithm:** Develop a stream-based join algorithm in Python that can efficiently join fast-arriving transactional data with large master data tables.
3. **ETL Pipeline Development:** Build a complete Extract, Transform, and Load (ETL) process that enriches transactional data with master data information.
4. **OLAP Analysis:** Implement 20 different analytical queries covering various business scenarios including revenue analysis, customer demographics, product performance, and temporal trends.
5. **Performance Optimization:** Ensure the system can handle near-real-time data processing requirements while maintaining data integrity.

1.3 System Architecture

The system follows a traditional data warehousing architecture with the following components:

- **Data Sources:** Three CSV files containing customer master data, product master data, and transactional data
- **ETL Layer:** Python-based HYBRIDJOIN implementation for data transformation and loading
- **Data Warehouse:** MySQL database implementing a star schema design
- **Analytical Layer:** SQL queries for OLAP operations including slicing, dicing, drill-down, and rollup

2. Data Warehouse Schema

2.1 Schema Design Approach

The project implements a **Star Schema** design, which is a widely-used dimensional modeling technique for data warehouses. The star schema provides an easily implemented model for multidimensional data analysis while preserving relational database structures.

2.2 Dimension Tables

2.2.1 Dim_Customer

The customer dimension table stores demographic and geographic information about customers:

- **Primary Key:** customer_id (INT)
- **Attributes:**
 - gender (VARCHAR(10)): Customer gender
 - age (VARCHAR(10)): Customer age group

- occupation (VARCHAR(50)): Customer occupation
- city_category (CHAR(1)): City classification (A, B, C)
- stay_in_current_city_years (VARCHAR(5)): Duration of stay in current city
- marital_status (BOOLEAN): Marital status indicator

This dimension enables analysis of customer behavior patterns based on demographics and location.

2.2.2 Dim_Product

The product dimension contains information about products sold:

- **Primary Key:** product_id (VARCHAR(20))
- **Attributes:**
 - product_category (VARCHAR(50)): Product category classification
 - price (DECIMAL(10,2)): Product price
 - store_id (INT): Foreign key to dim_store
 - supplier_id (INT): Foreign key to dim_supplier

The product dimension links to store and supplier dimensions, enabling hierarchical analysis of product performance across different stores and suppliers.

2.2.3 Dim_Store

The store dimension provides information about retail locations:

- **Primary Key:** store_id (INT)
- **Attributes:**
 - store_name (VARCHAR(100)): Name of the store

This dimension supports store-level performance analysis and comparisons.

2.2.4 Dim_Supplier

The supplier dimension tracks supplier information:

- **Primary Key:** supplier_id (INT)
- **Attributes:**
 - supplier_name (VARCHAR(100)): Name of the supplier

This dimension enables supplier performance analysis and supply chain insights.

2.2.5 Dim_Date

The date dimension is a critical component for temporal analysis:

- **Primary Key:** date_id (INT) - Format: YYYYMMDD
- **Attributes:**
 - full_date (DATE): Complete date value
 - day (INT): Day of month
 - month (INT): Month number (1-12)
 - quarter (INT): Quarter number (1-4)
 - year (INT): Year value
 - weekday (VARCHAR(10)): Day of week name

The date dimension supports various time-based analyses including seasonal trends, quarterly comparisons, and weekday vs. weekend analysis.

2.3 Fact Table

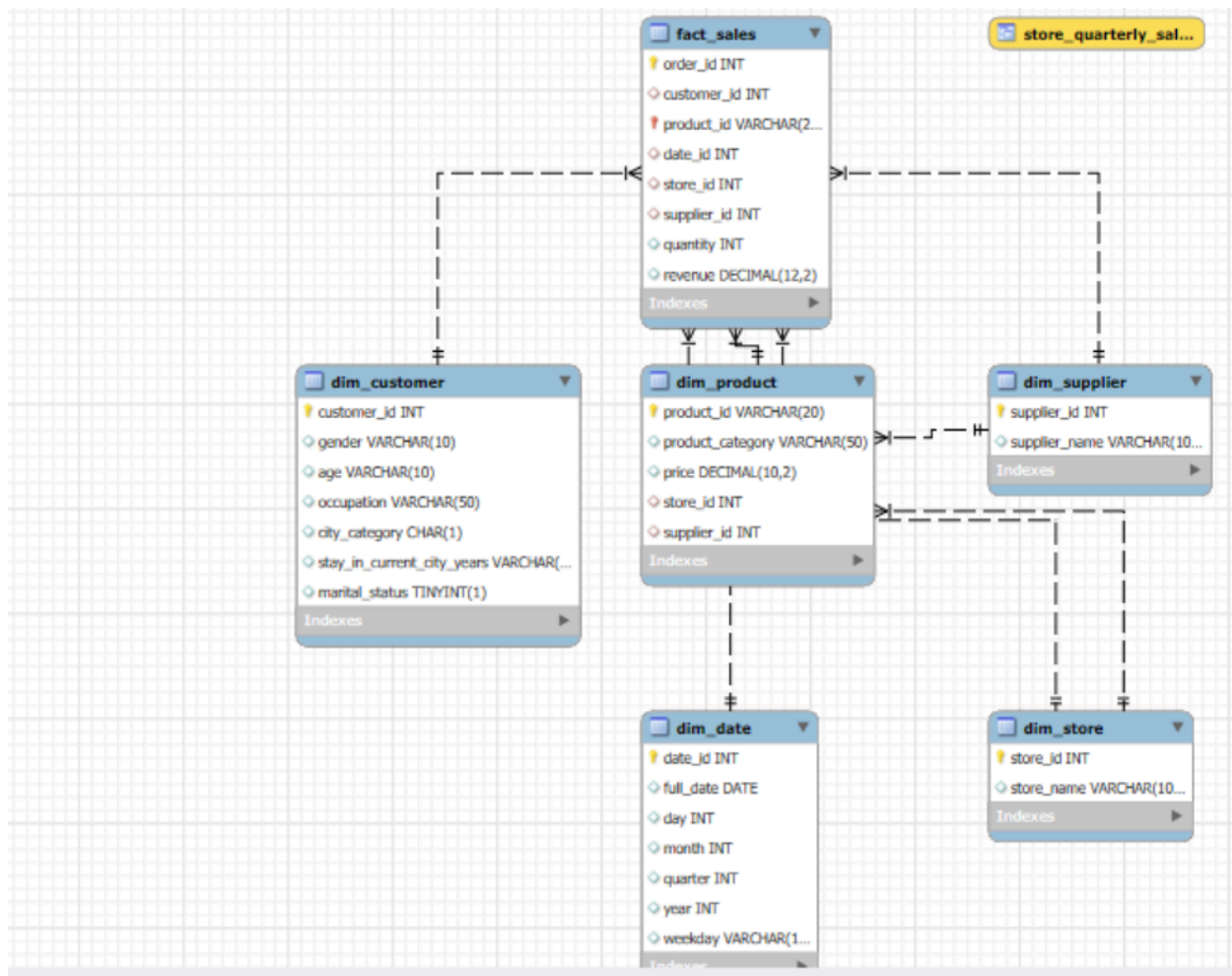
2.3.1 Fact_Sales

The fact table stores transactional sales data:

- **Composite Primary Key:** (order_id, product_id)
- **Foreign Keys:**
 - customer_id → dim_customer
 - product_id → dim_product
 - date_id → dim_date
 - store_id → dim_store
 - supplier_id → dim_supplier
- **Measures:**
 - quantity (INT): Quantity of products sold
 - revenue (DECIMAL(12,2)): Total revenue for the transaction line

The fact table follows the grain of one row per order-product combination which allows detailed analysis of individual transactions while supporting aggregation at various levels.

2.4 Schema Diagram



2.5 Design Rationale

The star schema design was chosen for several reasons:

1. **Query Performance:** Star schemas provide better query performance for analytical workloads compared to normalized schemas, as they reduce the number of joins required.
2. **Simplicity:** The design is easy to understand which makes it accessible to business users and analysts.

3. **Scalability:** The dimensional model can easily accommodate additional dimensions or measures as business requirements evolve.
4. **OLAP Support:** The structure naturally supports OLAP operations such as rollup, drill-down, slicing, and dicing.

3. HYBRIDJOIN Algorithm Explanation

3.1 Algorithm Overview

HYBRIDJOIN is a stream-based join algorithm designed for near-real-time data warehousing scenarios where fast-arriving, potentially bursty transactional data streams need to be joined with large, disk-based master data relations. The algorithm efficiently handles the challenge of joining a continuous data stream (S) with a large relation (R) stored on disk.

3.2 Key Components

3.2.1 Stream Buffer

A thread-safe queue that temporarily holds incoming stream tuples when the algorithm cannot process them immediately. This prevents data loss during bursty scenarios when the arrival rate exceeds processing capacity.

Configuration: Maximum size of 20,000 entries ($2 \times$ hash table slots)

3.2.2 Hash Table (H)

A multi-map data structure that stores stream tuples indexed by their join key (Customer_ID in this implementation). The hash table:

- Has a fixed number of slots ($hS = 10,000$) to limit memory usage
- Allows multiple entries per key (multi-map behavior)
- Each entry includes a pointer to a corresponding node in the queue
- Uses thread-safe locks for concurrent access

3.2.3 Doubly-Linked Queue

A FIFO queue implemented as a doubly-linked list that:

- Maintains the order of arrival for stream tuples
- Enables efficient random deletions when tuples are matched
- Tracks timestamps for time-based eviction of stale entries
- Supports thread-safe operations

3.2.4 Disk Buffer

A memory buffer that holds a loaded partition of the master data relation (R). Each partition contains 500 tuples ($vP = 500$), representing the “Join Window” for processing.

3.2.5 Available Slots Counter (w)

Tracks the number of free slots in the hash table, equal to the number of vacated spaces from the previous iteration. This determines how many new stream tuples can be loaded in each cycle.

3.3 Algorithm Workflow

The HYBRIDJOIN algorithm operates in the following steps:

Step 1: Initialization

- Initialize hash table with hS slots (10,000)
- Initialize empty queue and disk buffer
- Set available slots (w) to hS
- Load dimension tables into the database

Step 2: Stream Producer Thread

A separate thread continuously reads from the transactional CSV file and places tuples into the stream buffer. This thread: - Reads data in chunks (200 rows at a time) for efficiency - Handles file I/O independently of the join operation - Signals completion when the entire file is processed

Step 3: Load Stream Tuples

The main algorithm loop: - Checks the stream buffer for new tuples - Loads up to w tuples into the hash table - Hashes each tuple's join key (Customer_ID) to determine the slot - Adds the tuple to the hash table bucket - Appends the join key to the queue - Decrements w for each loaded tuple

Step 4: Load Disk Partition

- Retrieves the oldest key from the queue (FIFO order)
- Uses `dim_customer` to load a relevant partition from the master data relation
- Loads up to vP (500) tuples into the disk buffer
- Leverages the relation's indexed and sorted nature to minimize I/O

Step 5: Probe and Join

For each tuple in the loaded disk partition: - Probes the hash table using the join key - For each match found: - Generates the join output (enriched transaction record) - Performs data transformation: - Looks up product information from in-memory cache - Converts date to `date_id` using date lookup table - Calculates revenue ($\text{price} \times \text{quantity}$) - Removes matched stream tuples from the hash table - Removes corresponding nodes from the queue - Increments w by the number of freed slots - Adds the enriched record to the insert buffer

Step 6: Batch Insert

When the insert buffer reaches the batch size (500 records): - Executes batch INSERT IGNORE into `fact_sales` - Commits the transaction - Clears the buffer

Step 7: Repeat

The algorithm loops back to Step 3, continuing until: - The stream is finished (producer thread signals completion) - The stream buffer is empty - The hash table is empty - The queue is empty

Step 8: Cleanup

- Flushes any remaining records in the insert buffer
- Closes database connections
- Reports processing statistics

3.4 Threading Model

The implementation uses a multi-threaded architecture:

1. **Producer Thread:** Reads transactional data from CSV and feeds the stream buffer
2. **Consumer Thread:** Executes the HYBRIDJOIN algorithm and loads data into the warehouse

This separation allows the system to handle I/O-bound operations (file reading) and CPU-bound operations (join processing) concurrently, improving overall throughput.

3.5 Data Enrichment Process

During the join operation, the algorithm enriches transactional data with:

1. **Customer Demographics:** From dim_customer (gender, age, occupation, city category, etc.)
2. **Product Information:** From in-memory product lookup (category, price, store, supplier)
3. **Date Attributes:** Converts date strings to date_id for temporal analysis
4. **Revenue Calculation:** Computes revenue as price × quantity

This enrichment transforms raw transactional data into analysis-ready fact table records.

3.6 Performance Optimizations

Several optimizations are implemented to improve performance:

1. **Batch Processing:** Inserts are batched (500 records) to reduce database round-trips
2. **In-Memory Caching:** Product master data is cached in memory for fast lookups
3. **Chunked File Reading:** CSV files are read in chunks to manage memory usage
4. **Thread-Safe Operations:** All shared data structures use locks to prevent race conditions
5. **TTL Eviction:** Stale entries in the hash table are evicted after 24 hours to prevent unbounded growth

4. Shortcomings of HYBRIDJOIN Algorithm

While HYBRIDJOIN is effective for near-real-time data warehousing, it has several limitations:

4.1 Memory Constraint Limitations

Shortcoming: The hash table has a fixed size (10,000 slots), which can become a bottleneck during high-volume data bursts. When the stream arrival rate exceeds the processing rate for extended periods, the hash table can fill up, causing the stream buffer to back up. Even with the stream buffer acting as a safety net, this can lead to:

- Increased memory consumption as the stream buffer grows
- Potential data loss if the stream buffer also reaches capacity
- Degraded performance as the system struggles to catch up

Impact: In production environments with highly variable data arrival rates, the fixed memory allocation may not be sufficient, requiring manual tuning or dynamic resizing capabilities.

4.2 Sequential Partition Processing

Shortcoming: The algorithm processes disk partitions sequentially based on the FIFO queue order. This means that even if a partition contains many matches, the algorithm must wait to process the next partition until the current one is complete. This sequential processing can be inefficient when:

- Some partitions have very few or no matches
- The distribution of join keys is uneven across partitions
- Multiple partitions could be processed in parallel

Impact: The sequential nature limits the algorithm's ability to fully utilize available I/O bandwidth and CPU resources, potentially reducing overall throughput in scenarios with high parallelism potential.

4.3 Limited Handling of Late-Arriving Data

Shortcoming: The algorithm uses a Time-To-Live (TTL) mechanism to evict stale entries after 24 hours. However, this approach has limitations:

- If a master data record arrives after its corresponding stream tuple has been evicted, the join cannot be completed
- The TTL is fixed and doesn't adapt to actual data arrival patterns
- There's no mechanism to reprocess evicted tuples when master data becomes available

Impact: In scenarios where master data updates are delayed or stream data arrives out of order, some valid joins may be missed, leading to incomplete data in the warehouse and potential data quality issues.

5. Lessons Learned

This project provided insights into data warehousing, stream processing, and algorithm implementation:

5.1 Dimensional Modeling

Working with star schemas reinforced the importance of proper dimensional modeling. The choice of dimensions and the grain of the fact table significantly impact both query performance and analytical capabilities. Understanding the business requirements and designing the schema accordingly is crucial for a successful data warehouse.

5.2 Performance Optimization

The project demonstrated several performance optimization techniques:

- **Batch Processing:** Grouping database operations significantly improves throughput
- **Caching:** In-memory caching of frequently accessed data reduces I/O operations
- **Indexing:** Proper indexing of dimension tables is essential for efficient joins

5.3 Data Quality Considerations

During implementation, several data quality challenges emerged:

- **Missing Data:** Handling cases where master data references don't exist
- **Data Type Consistency:** Ensuring consistent data types across different data sources
- **Date Formatting:** Standardizing date formats for temporal analysis

5.4 Real-World Application

This project provided experience with:

- **ETL Pipeline Development:** Building end-to-end data pipelines from source to warehouse

- **Database Design:** Creating and managing relational database schemas
- **SQL Query Optimization:** Writing efficient analytical queries
- **Python Programming:** Implementing complex algorithms with error handling

6. Conclusion

This project successfully implemented a near-real-time data warehouse prototype for Walmart using the HYBRIDJOIN algorithm. The system demonstrates the feasibility of processing high-volume transactional data streams while enriching them with master data and loading them into a dimensional data warehouse.